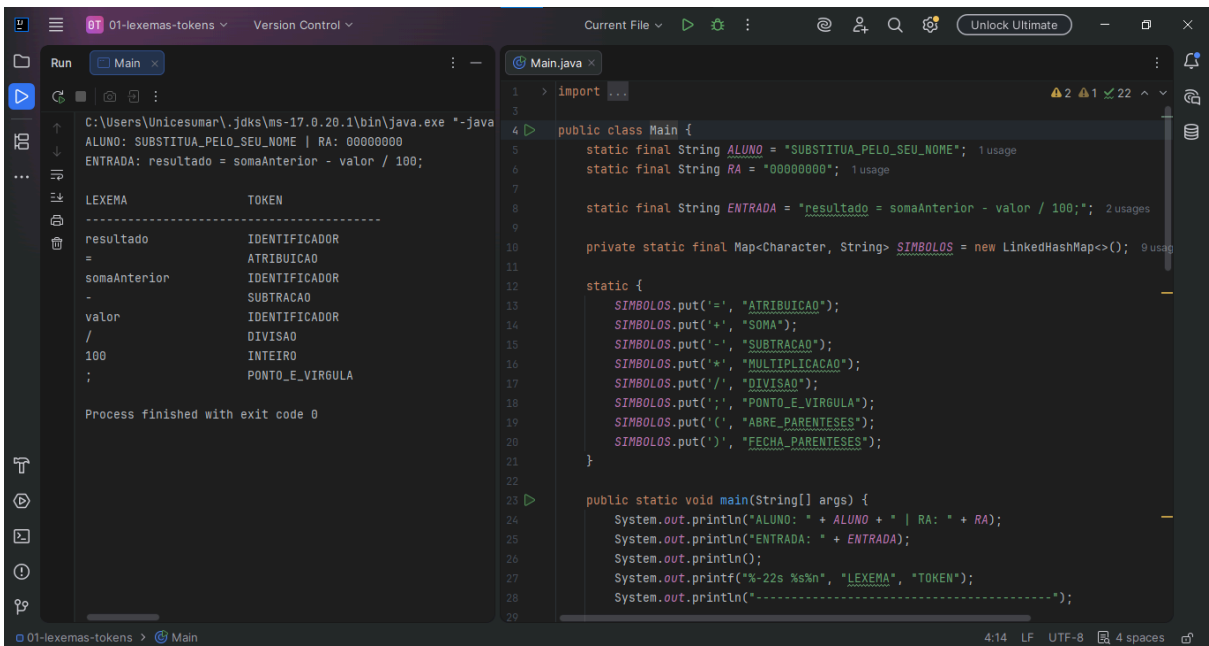
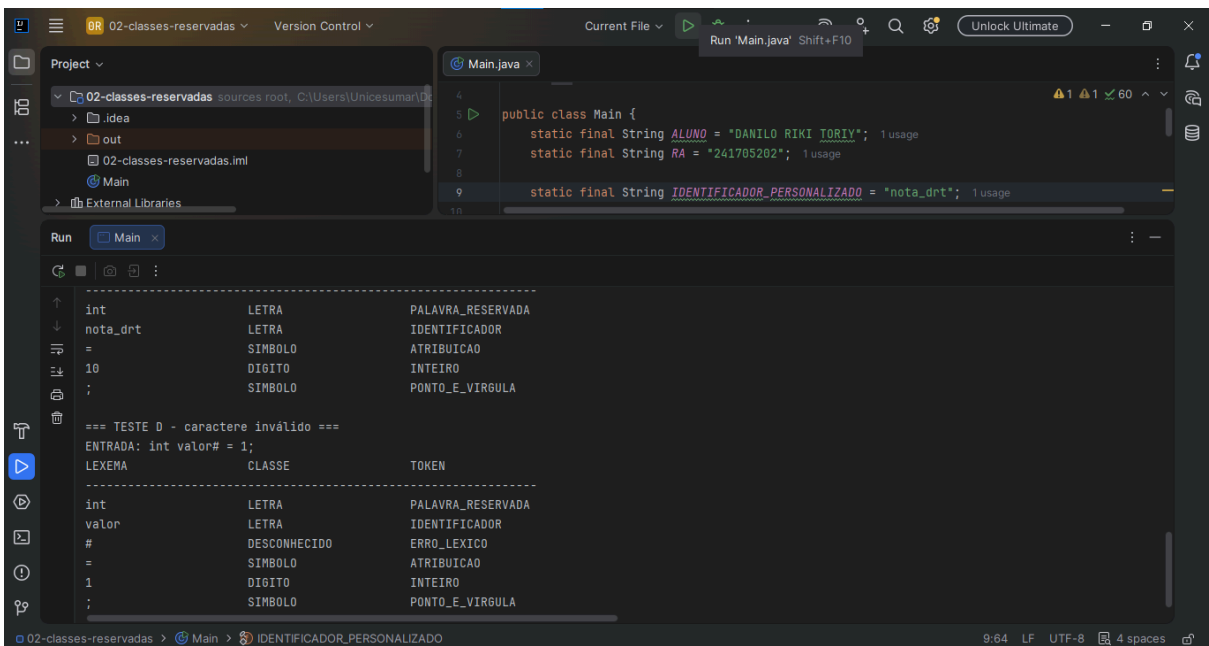


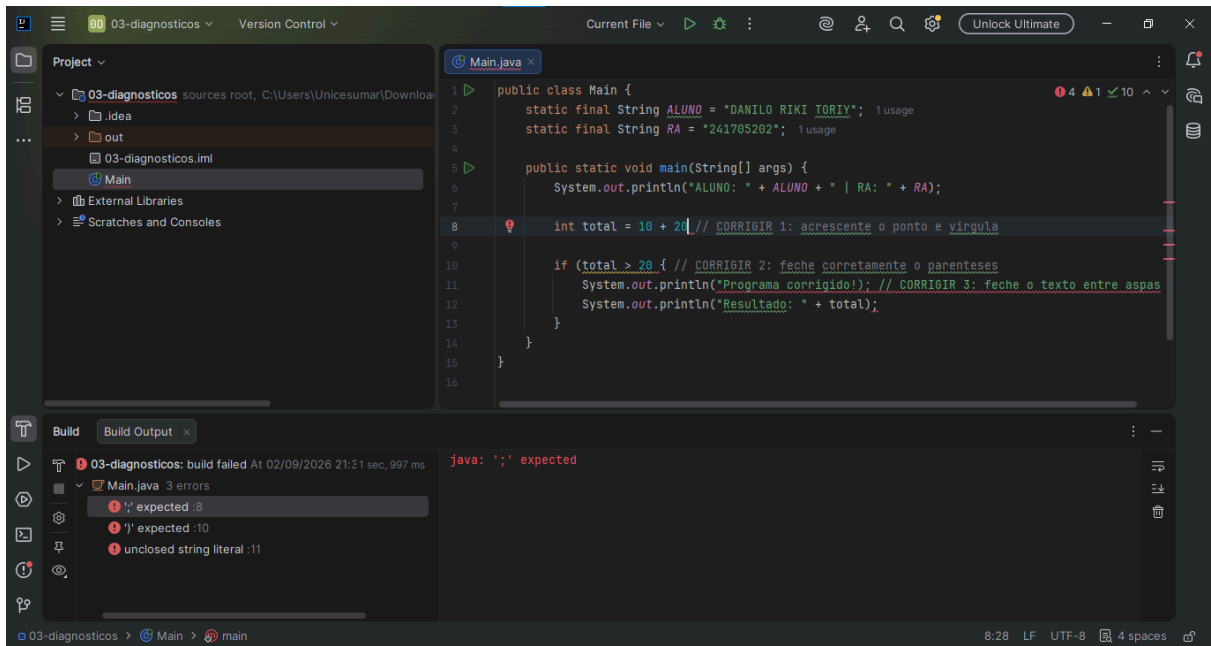
Evidência exercício 1:



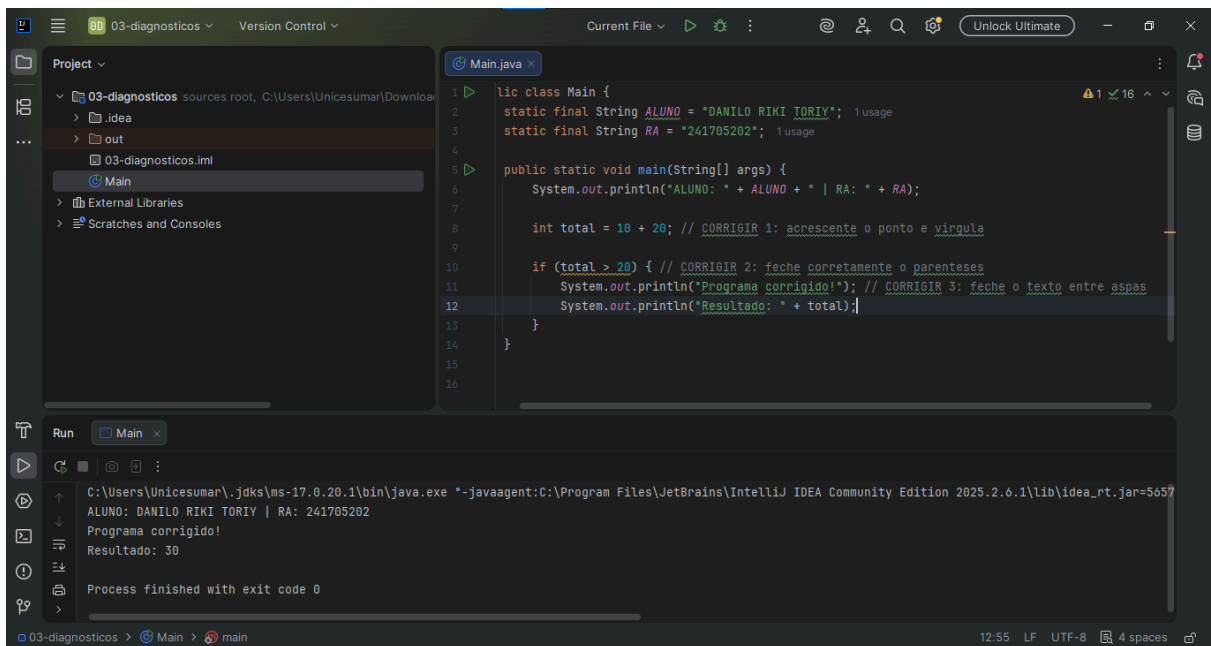
Evidência Exercício 2:



Evidência Exercício 3 - Erro:



Evidência Exercício 3 - Corrigido:



Evidência Exercício 4 - 2 prints para a saída do terminal no intelliJ:

04-parser-descendente

Version Control

Current File

Unlock Ultimate

Run

Main

C:\Users\Unicesumar\.jdk\ms-17.0.20.1\bin\java.exe

ALUNO: DANILO RIKI TORIY | RA: 241705202

ENTRADA: (2 + 3) * 4

ENTRANDO em <expr> | lookahead = (
ENTRANDO em <term> | lookahead = (
ENTRANDO em <factor> | lookahead = (
Abrindo parenteses
Novo lookahead: 2
ENTRANDO em <expr> | lookahead = 2
ENTRANDO em <term> | lookahead = 2
ENTRANDO em <factor> | lookahead = 2
Numero reconhecido: 2
Novo lookahead: +
SAINDO de <factor> | lookahead = +
SAINDO de <term> | lookahead = +
Encontrado operador +
Novo lookahead: 3
ENTRANDO em <term> | lookahead = 3
ENTRANDO em <factor> | lookahead = 3
Numero reconhecido: 3
Novo lookahead:)
SAINDO de <factor> | lookahead =)
SAINDO de <term> | lookahead =)
SAINDO de <expr> | lookahead =)
Novo lookahead: *

Main.java

1 public class Main {
2 static final String ALUNO = "DANILO RIKI TORIY"; 1 usage
3 static final String RA = "241705202"; 1 usage
4
5 static final String ENTRADA = "(2 + 3) * 4"; 2 usages
6 static final boolean DEMONSTRAR_RECUSAO_ESQUERDA = false; 1 usage
7
8 public static void main(String[] args) {
9 System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
10 System.out.println("ENTRADA: " + ENTRADA);
11 System.out.println();
12
13 if (DEMONSTRAR_RECUSAO_ESQUERDA) {
14 demonstrarRecusaoEsquerda(0);
15 System.out.println();
16 }
17
18 try {
19 Parser parser = new Parser(ENTRADA);
20 int resultado = parser.analisar();
21 System.out.println();
22 System.out.println("ENTRADA ACEITA");
23 System.out.println("RESULTADO DA EXPRESSAO: " + resultado);
24 } catch (ErroSintatico erro) {
25 System.out.println();
26 System.out.println("ERRO SINTATICO: " + erro.getMessage());
27 }
28 }

04-parser-descendente > Main > main 8:44 LF UTF-8 4 spaces

```
Run Main x
SAINDO de <factor> | lookahead = +
SAINDO de <term> | lookahead = +
Encontrado operador +
Novo lookahead: 3
ENTRANDO em <term> | lookahead = 3
ENTRANDO em <factor> | lookahead = 3
Numero reconhecido: 3
Novo lookahead: )
SAINDO de <factor> | lookahead = )
SAINDO de <term> | lookahead = )
SAINDO de <expr> | lookahead = )
Novo lookahead: *
Fechando parenteses
SAINDO de <factor> | lookahead = *
Encontrado operador *
Novo lookahead: 4
ENTRANDO em <factor> | lookahead = 4
Numero reconhecido: 4
Novo lookahead: $FIM$
SAINDO de <factor> | lookahead = $FIM$
SAINDO de <term> | lookahead = $FIM$
SAINDO de <expr> | lookahead = $FIM$

ENTRADA ACEITA
RESULTADO DA EXPRESSAO: 20
```

Evidência Exercício 5:

05-parser-ascendente

Version Control

Current File

Unlock Ultimate

Run

Main

C:\Users\Unicesumar\.jdk\ms-17.0.20.1\bin\java.exe -javaagent:C:\Program Files\ALUNO: DANILO RIKI TORIV | RA: 241705202
ENTRADA: id+id*id

PILHA	ENTRADA	ACAO
\$ 0	id+id*id\$	SHIFT id -> estado 5
\$ 0 id 5	+id*id\$	REDUCE F -> id
\$ 0 F 3	+id*id\$	REDUCE T -> F
\$ 0 T 2	+id*id\$	REDUCE E -> T
\$ 0 E 1	+id*id\$	SHIFT + -> estado 6
\$ 0 E 1 + 6	id*id\$	SHIFT id -> estado 5
\$ 0 E 1 + 6 id 5	*id\$	REDUCE F -> id
\$ 0 E 1 + 6 F 3	*id\$	REDUCE T -> F
\$ 0 E 1 + 6 T 9	*id\$	SHIFT * -> estado 7
\$ 0 E 1 + 6 T 9 * 7	id\$	SHIFT id -> estado 5
\$ 0 E 1 + 6 T 9 * 7 id 5	\$	REDUCE F -> id
\$ 0 E 1 + 6 T 9 * 7 F 10	\$	REDUCE T -> T * F
\$ 0 E 1 + 6 T 9	\$	REDUCE E -> E + T
\$ 0 E 1	\$	ACCEPT

Process finished with exit code 0

Main.java

```
1 > import ...
5
6 public class Main {
7     static final String ALUNO = "DANILO RIKI TORIV"; 1 usage
8     static final String RA = "241705202"; 1 usage
9
10    static final String ENTRADA_VALIDA = "id+id*id"; 1 usage
11    static final String ENTRADA_INVALIDA = "id*+id"; no usages
12
13    // Na primeira execução, use ENTRADA_VALIDA. Depois troque
14    static final String ENTRADA = ENTRADA_VALIDA; 2 usages
15
16    static final Map<Integer, Map<String, String>> ACTION = new
17    static final Map<Integer, Map<String, Integer>> GOTO = new
18    static final Map<Integer, Regra> REGRAS = new HashMap<>();
19
20    static {
21        // Gramática:
22        // 1) E -> E + T 2) E -> T
23        // 3) T -> T * F 4) T -> F
24        // 5) F -> ( E ) 6) F -> id
25        regra( numero: 1, esquerda: "E", tamanhoDireita: 3, descricao:
26        regra( numero: 2, esquerda: "E", tamanhoDireita: 1, descricao:
27        regra( numero: 3, esquerda: "T", tamanhoDireita: 3, descricao:
28        regra( numero: 4, esquerda: "T", tamanhoDireita: 1, descricao:
29        regra( numero: 5, esquerda: "F", tamanhoDireita: 3, descricao:
30        regra( numero: 6, esquerda: "F", tamanhoDireita: 1, descricao:
31    }
```

05-parser-ascendente > Main > RA 8:40 LF UTF-8 4 spaces